# HookLens

웹훅을 받아서 실시간으로 보여주고, 정해 둔 응답을 돌려주는 웹훅 디버거

GITHUB https://github.com/songjeongseon1/hooklens

| | |
|---|---|
| 기간 | 2026.10.13 ~ 2026.10.22 (10일, 커밋 26개 · PR 8개) |
| 인원 | 1인 — 기획, 백엔드, 프론트엔드, 배포 구성 |
| 저장소 | https://github.com/songjeongseon1/hooklens |
| 기술 | Java 21, Spring Boot 4.0, Spring Data JPA, MySQL 8 / React 19, Vite 8, Tailwind CSS 4 / JUnit 5, Testcontainers / Docker Compose, Nginx |
| 규모 | 백엔드 1,654줄(38파일), 테스트 1,260줄(59개), 프론트엔드 742줄(16파일) |

## 1 개요

### 배경

결제(토스페이먼츠 등)나 깃허브 연동을 만들 때는 외부 서비스가 우리 서버로 보내는 웹훅을 받아서 처리하는 코드를 짜야 한다. 이때 반복해서 막히는 지점이 세 가지 있었다.

- 실제로 어떤 헤더와 바디가 오는지 문서만으로는 확신하기 어렵다.
- 외부 서비스는 인터넷에 있어서 개발 중인 PC(localhost)로는 웹훅을 보낼 수 없다.
- 우리 서버가 실패하거나 느리게 응답할 때 상대가 재전송하는지 미리 확인할 방법이 없다.

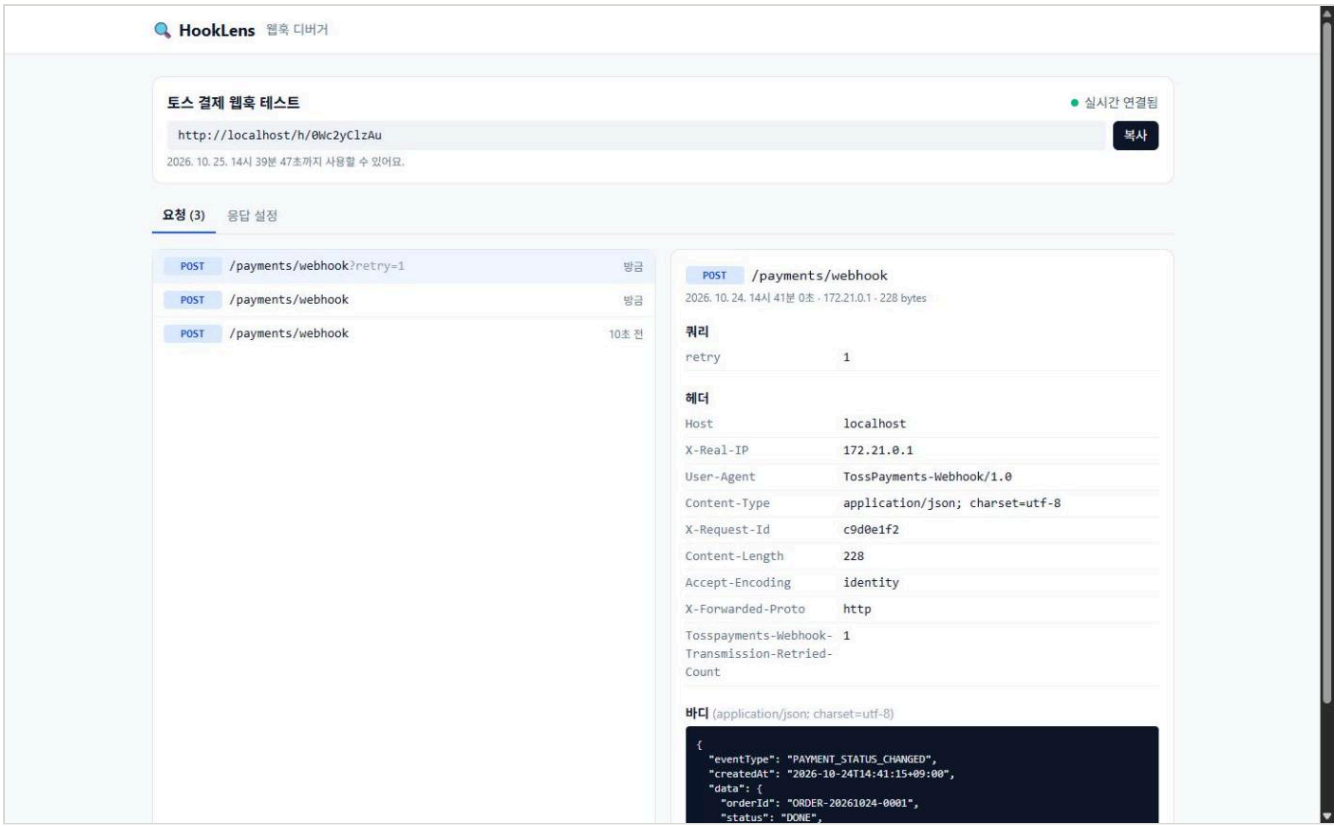
그래서 인터넷에서 접근 가능한 주소를 발급하고, 들어온 요청을 가공하지 않고 기록해 화면에 바로 띄우며, 응답 코드와 지연을 바꿔 실패 상황을 재현할 수 있는 도구를 만들었다. webhook.site, RequestBin과 같은 종류다.

### 목표와 범위

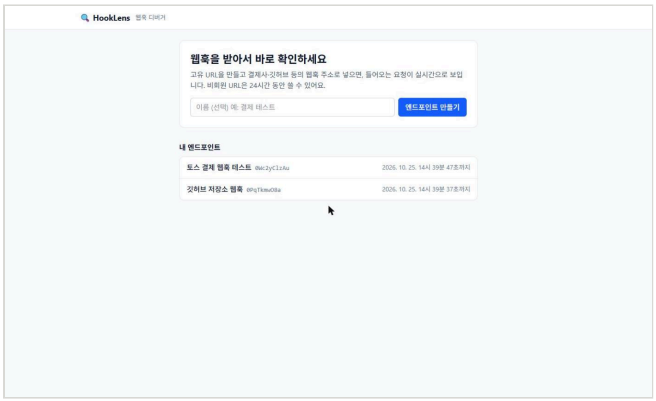
2주 안에 MVP(P0 8개)를 완성하고 배포 구성까지 마치는 것을 목표로 잡았다. 회원·요청 제한·리플레이는 2차로 미뤘다.

| 요구사항 | 내용 |
|---|---|
| EP-001, EP-002 | 비회원 엔드포인트 발급, 24시간 후 만료(이후 수신 시 410) |
| HOOK-001, HOOK-002 | 모든 메서드·하위 경로 수신 및 원본 저장, 설정된 응답 반환(없으면 200 `{"ok":true}`) |
| LOG-001, 002, 003 | SSE 실시간 반영, 최신순 커서 목록, 상세(쿼리·헤더·바디) |
| RES-001 | 기본 응답 설정(상태 코드, 헤더, 바디, 지연 0~5초) |

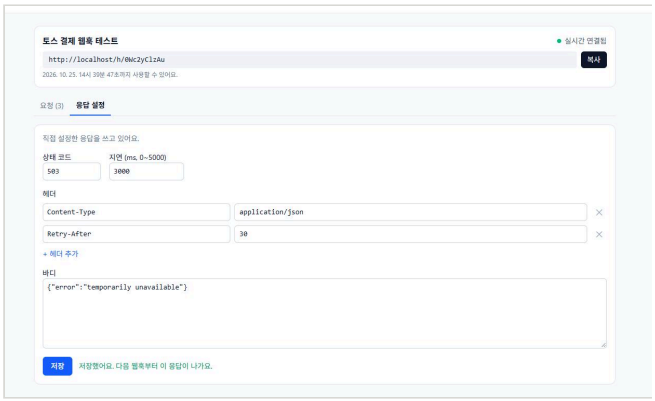
## 2 화면과 기능




엔드포인트 화면. 결제 웹훅 3건이 새로고침 없이 들어왔고, 재전송 요청( `retry=1` )의 헤더와 바디를 보고 있다.



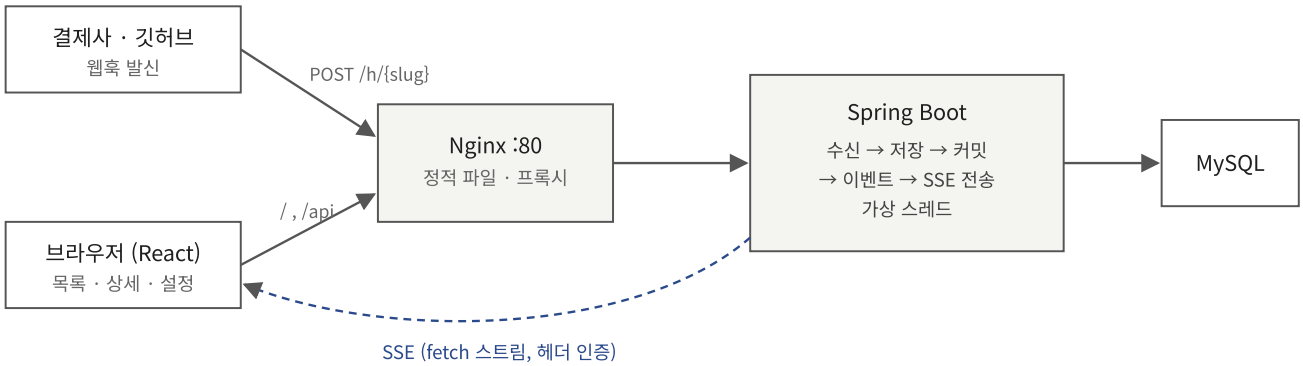

홈. 이름만 넣으면 주소가 발급되고, 이 브라우저에서 만든 엔드포인트가 쌓인다.




응답 설정. 503 + 3초 지연으로 바꾸면 다음 웹훅부터 그대로 응답한다.

- 로그인 없이 쓰는 대신, 발급 응답에서 한 번만 주는 익명 토큰을 브라우저(localStorage)에 보관하고 관리 API마다 `X-Anonymous-Token` 헤더로 보낸다.
- 요청은 메서드·경로·쿼리·헤더(같은 이름이 여러 번 오면 모두)·바디·IP·수신 시각을 원본 그대로 남긴다.

## 3 시스템 구조




- 패키지는 기능 단위(`endpoint`, `hook`, `log`, `response`, `global`)로 나눴다.
- 관리 API는 `{ success, data, error: { code, message } }`로 응답하고, 에러 코드는 `ErrorCode` enum 한 곳에서 관리한다. 웹훅 수신(`/h/**`)만은 사용자가 정한 응답을 그대로 돌려준다.
- 모든 계층의 시간을 Asia/Seoul로 맞추고 API에는 오프셋을 붙여 내보낸다(`2026-10-22T17:11:52+09:00`).

### 데이터 모델

| 테이블 | 주요 컬럼 | 비고 |
|---|---|---|
| endpoint | slug(UK, 영숫자 10자), anonymous_token_hash(SHA-256), expires_at, member_id(NULL=비회원) | 외부에는 숫자 id 대신 slug만 노출 |
| request_log | method, path, query_string, headers(JSON), body(MEDIUMTEXT), ip, size_bytes, received_at | 인덱스 (endpoint_id, request_id DESC) |
| response_rule | is_default, status_code, response_headers(JSON), response_body, delay_ms | 1차는 엔드포인트당 기본 규칙 1개 |

엔드포인트가 지워지면 로그와 응답 규칙은 FK의 ON DELETE CASCADE로 함께 지워진다.

### API

| 메서드 | 경로 | 설명 |
|---|---|---|
| POST | `/api/endpoints` | 발급. 익명 토큰은 이 응답에서만 제공 |
| ANY | `/h/{slug}/**` | 웹훅 수신. 없는 slug 404, 만료 410 |
| GET | `/api/endpoints/{slug}/requests` | 목록. `cursor`, `size`(1~100) |
| GET | `/api/endpoints/{slug}/requests/{id}` | 상세. 헤더는 JSON 객체로 변환 |
| GET | `/api/endpoints/{slug}/stream` | SSE. `connected`, `request` 이벤트, 30초 하트비트 |
| GET/PUT | `/api/endpoints/{slug}/response` | 응답 설정 조회 / 저장(upsert) |

## 4 기술적으로 고민한 부분

### 4.1 SSE 인증 — EventSource를 쓰지 않은 이유

**문제** 관리 API는 모두 `X-Anonymous-Token` 헤더로 인증하는데, 브라우저의 `EventSource`는 커스텀 헤더를 보낼 수 없다.

**검토** 쿼리 파라미터로 토큰 전달 → Nginx 접근 로그와 브라우저 기록에 24시간짜리 토큰이 남는다. 일회용 티켓 발급 → 티켓 저장소와 만료 처리, 재연결 때마다 재발급이 필요하다. 쿠키 → 인증 방식이 둘로 갈리고 CSRF를 따로 고려해야 한다.

**선택** `fetch`로 스트림을 열고 `response.body.getReader()`로 직접 읽는 훅을 만들었다. 네트워크 조각은 줄 단위로 오지 않으므로 버퍼에 모아 줄로 자르고(마지막 덜 온 조각은 다음 조각과 합침), 빈 줄에서 메시지 하나를 완성하고, `:`로 시작하는 하트비트는 버린다.

**결과** 인증 방식이 하나로 통일됐고 백엔드는 다른 관리 API와 똑같이 헤더를 읽는다. 대신 EventSource가 해 주던 재연결을 직접 구현했다 — 스트림이 끝나면 3초 뒤 다시 연결하고, 끊긴 사이 놓친 요청은 목록 API로 다시 받아 requestId로 중복을 없앤다. 401·403·410은 재시도해도 같으므로 다시 연결하지 않는다.

### 4.2 알림은 커밋이 끝난 뒤에

**문제** 웹훅을 저장하면서 바로 SSE로 알리면, 화면이 알림을 받고 상세를 조회했을 때 아직 커밋 전이라 404가 날 수 있다. 저장이 롤백되면 없는 요청을 알린 셈이 된다.

**선택** 수신 서비스는 저장 후 `RequestReceivedEvent`만 발행하고, 리스너를 `@TransactionalEventListener(phase = AFTER_COMMIT)`로 두어 커밋이 성공했을 때만 전송한다. 이벤트에는 엔티티 대신 요약 DTO를 담았다 — 리스너는 트랜잭션 밖에서 돌기 때문에 LAZY 필드를 건드리면 예외가 난다.

**결과** 수신 코드는 SSE의 존재를 모른다. 서버를 여러 대로 늘려 Redis Pub/Sub로 바꿔도 수신 코드는 그대로다. 실제 포트로 서버를 띄운 E2E 테스트에서 리스너의 전송 줄을 지우면 테스트가 실패하는 것까지 확인했다.

### 4.3 지연 응답과 커넥션 풀, 가상 스레드

**문제** 응답 지연(최대 5초)을 서비스의 트랜잭션 안에서 기다리면 그동안 DB 커넥션을 잡고 있다. HikariCP 기본 풀은 10개라 지연 요청 10개면 지연과 무관한 요청까지 멈춘다. 또 기다리는 동안 톰캣 스레드(기본 200개)도 묶인다.

**검토** 그냥 `Thread.sleep` / `DeferredResult`와 스케줄러로 비동기 완성 / 가상 스레드. 비동기는 스레드를 안 묶지만 흐름이 복잡해진다.

**선택** 로그 저장과 규칙 조회까지만 트랜잭션으로 묶어 먼저 커밋하고, 대기는 컨트롤러에서 한다. 그리고 `spring.threads.virtual.enabled`로 가상 스레드를 켰다. 코드는 단순한 sleep이지만 기다리는 동안 OS 스레드를 놓아준다.

**결과** 지연 중에도 로그는 이미 커밋돼 화면에 먼저 뜬다. 이를 확인하는 테스트(1.5초 지연 요청을 보내 두고, 응답이 오기 전에 DB에 로그가 있는지 검사)를 두었고, 실행 중 요청 스레드 이름이 `tomcat-handler-N`인 것으로 가상 스레드 적용을 확인했다. 같은 이유로 `open-in-view`를 꺼 두어 30분짜리 SSE 연결도 커넥션을 잡지 않는다.

### 4.4 새 데이터가 계속 들어오는 목록의 페이지네이션

**문제** 목록을 보는 중에도 웹훅이 들어온다. 오프셋 방식이면 앞에 새 행이 끼면서 페이지가 밀려 같은 요청이 두 번 보이거나 빠지고, 뒤 페이지로 갈수록 앞의 행을 세고 버려 느려진다.

**선택** `WHERE endpoint_id = ? AND request_id < :cursor ORDER BY request_id DESC` 커서 방식으로 하고, 같은 순서의 복합 인덱스를 두었다. 다음 페이지 여부는 COUNT 쿼리 대신 `size + 1` 개를 조회해 판단하고, 첫 페이지는 cursor를 `Long.MAX_VALUE` 로 둬서 메서드 하나로 처리했다. Spring Data의 `Pageable` 은 OFFSET을 쓰므로 `Limit` 을 썼다.

### 4.5 인증과 접근 제어

**토큰** 익명 토큰은 32바이트 난수(Base64URL)로 만들고 DB에는 SHA-256 해시만 저장한다. BCrypt는 추측 가능한 비밀번호를 느리게 만들어 막는 용도라, 무차별 대입이 불가능한 난수 토큰에는 빠른 해시로 충분하고 바로 조회할 수 있다고 판단했다. 비교는 `equals` 대신 `MessageDigest.isEqual` 로 해서 응답 시간으로 일치 길이를 추측하는 타이밍 공격을 피했다.

**IDOR** 주인 확인은 "주소의 slug가 내 것인가"만 본다. 그래서 내 토큰으로 `/api/endpoints/{내 slug}/requests/{남의 id}` 를 요청하면 통과해 버린다. 상세 조회를 `findByIdAndEndpointId` 로 바꿔 남의 요청은 조회 결과에 아예 나오지 않게 했고, 존재 여부를 숨기려고 403이 아닌 404를 준다. 존재하지 않는 id와 남의 id가 같은 응답이라는 것도 테스트로 고정했다.

**상태 코드** 토큰이 없으면 401, 있는데 남의 것이면 403, 잘못된 파라미터( `size=0` , `cursor=abc` )는 500이 아닌 400이 되도록 예외 핸들러를 보강했다.

### 4.6 사용자가 정한 응답 헤더 — 헤더 주입

**문제** 응답 설정의 헤더는 웹훅 응답에 그대로 나간다. 값에 `\r\n` 이 있으면 `X-A: x\r\nSet-Cookie: ...` 처럼 헤더를 끼워 넣을 수 있다(HTTP 응답 분할). 또 `Content-Length` 를 실제와 다르게 넣으면 응답이 잘린다.

**선택** 헤더 이름은 HTTP 규격이 허용하는 문자만 받는 정규식(허용 목록)으로, 값은 줄바꿈만 막았다. 어느 헤더가 왜 틀렸는지 메시지에 담아 400으로 돌려준다. 전송 제어 헤더(Content-Length, Transfer-Encoding 등)는 저장은 하되 응답할 때 무시한다. 저장은 PUT 하나로 있으면 변경 감지로 UPDATE, 없으면 INSERT하는 upsert라 여러 번 눌러도 규칙은 하나다.

### 4.7 바디를 원본 그대로 읽기

**문제** 폼 형식( `application/x-www-form-urlencoded` )의 PUT 웹훅을 받으면 바디가 비어서 저장됐다.

**원인** Spring의 `FormContentFilter` 가 PUT·PATCH·DELETE 폼 바디를 먼저 읽어 파라미터로 바꿔 둔다. 바디 스트림은 한 번만 읽을 수 있어 컨트롤러에는 남은 게 없었다.

**해결** 관리 API는 JSON만 쓰므로 이 필터를 끄고, 수신부는 `@RequestBody` 나 `getParameter()` 없이 입력 스트림을 바이트로 직접 읽는다. 이 경우를 고정하는 테스트를 남겼다.

## 5 테스트

- 테스트 59개(단위 + 통합). 통합 테스트는 H2 대신 Testcontainers로 실제 MySQL을 띄운다 — JSON 컬럼, 인덱스, CASCADE가 운영과 같게 동작해야 의미가 있다고 봤다.
- SSE는 MockMvc로는 끝나지 않는 응답을 읽기 어려워, 실제 포트로 서버를 띄우고 자바 `HttpClient`로 스트림에 연결한 상태에서 웹훅을 보내 `event:request`가 도착하는지 확인했다. 무한 대기를 막으려고 5초 제한을 걸었다.
- 보안 관련 테스트는 실제로 검사 코드를 지웠을 때 실패하는지 확인했다(헤더 검사 호출을 빼면 2개 실패, 리스너 전송을 빼면 E2E 실패).
- 시간은 `Clock` 빈에서만 가져와서 "발급 24시간 뒤 만료"를 시간을 고정해 경계값(1초 전·정각·1초 후)으로 검증했다.

| 테스트 클래스 | 확인 내용 |
|---|---|
| EndpointApiTest 외 3개 | 발급, slug 형식·중복 재시도, 토큰 해시, 만료 경계 |
| HookReceiveApiTest | 원본 저장, 폼 PUT 바디, 같은 이름 헤더, 404/410, 규칙 응답, 지연, 지연 중 로그 선저장 |
| RequestLogApiTest | 커서 페이징(중복 없음), 다른 엔드포인트 분리, IDOR 404, 401/403/400 |
| RequestStreamApiTest, E2ETest | 구독·인증, 실제 스트림 수신 |
| ResponseSettingsApiTest | upsert 멱등, 실제 웹훅 반영, 범위 검증, 헤더 주입 차단 |

## 6 배포 구성

EC2 한 대에 Nginx, 백엔드, MySQL을 Docker Compose 하나로 띄우는 구성을 만들고, 로컬에서 같은 구성을 그대로 실행해 검증했다. RDS 대신 컨테이너 MySQL로 시작한 것은 일정과 비용 때문이며, 접속 주소만 바꾸면 옮길 수 있다.

- 밖으로는 Nginx의 80번만 연다. 백엔드(8080)와 MySQL(3306)은 Docker 내부망에서만 보인다.
- 백엔드·프론트 모두 2단계 빌드로, 실행 이미지에는 JRE + jar, Nginx + 빌드 결과만 담았다(프론트 이미지 74MB). 빌드 설정 파일을 먼저 복사해 의존성 층을 캐시한다.
- Nginx는 SSE 경로에만 `proxy_buffering off`와 1시간 read timeout을 주고, `/e/:slug`에서 새로고침해도 404가 나지 않게 `try_files ... /index.html`을 둔다.
- 운영은 `ddl-auto: validate`라 엔티티에서 생성한 DDL을 MySQL 첫 실행 때 넣는다. 백엔드는 MySQL healthcheck가 healthy가 된 뒤에 시작한다.
- Nginx 뒤에서 모든 IP가 Nginx 주소로 찍히는 문제는 `X-Forwarded-For` 전달과 `server.forward-headers-strategy: native`로 해결했다. 톰캣은 사설 대역에서 온 헤더만 믿어 외부 위조를 막는다.

## 7 한계와 다음 계획

- EC2 실제 배포는 하지 않았다. 구성과 절차(`docs/deploy.md`)는 준비돼 있다.
- 2차: 회원·JWT, 엔드포인트별 분당 요청 제한(Redis), 요청 리플레이와 SSRF 차단, 로그 보관 기간 정리 스케줄러, GitHub Actions 배포.
- SSE 연결 목록이 서버 메모리에 있어서 서버가 여러 대면 다른 서버의 연결에 알림이 가지 않는다 → Redis Pub/Sub.
- 스키마를 SQL 파일로 관리하고 있어 엔티티를 바꿀 때마다 다시 생성해야 한다 → Flyway.

- 바이너리 바디는 UTF-8 문자열로 저장해 깨질 수 있고, 같은 엔드포인트에 응답 설정 첫 저장이 동시에 오면 규칙이 두 개 생길 수 있다.

## 8 회고

기능마다 "무엇을 왜 이렇게 하는지"를 먼저 정리하고 코드를 쓰는 식으로 진행했다. 덕분에 커서 페이징, AFTER_COMMIT 리스너, IDOR처럼 처음엔 그냥 넘어갔을 부분을 직접 비교하고 고를 수 있었다.

가장 기억에 남는 건 설정 오타다. `threads`를 `thread`로, `forward-headers-strategy`를 `forword`로 쓴 적이 있는데, Spring은 모르는 키를 오류 없이 무시해서 앱은 잘 뜨고 기능만 조용히 빠진다. 그 뒤로는 설정을 바꾸면 실제 동작(스레드 이름, 저장된 IP)을 눈으로 확인하는 습관을 들였다. 같은 이유로 테스트도 "통과하는지"보다 "검사 코드를 지우면 실패하는지"를 확인하게 됐다.

아쉬운 점은 배포까지 마치지 못한 것이다. 일정 안에서 범위를 정할 때 프론트와 배포의 비중을 너무 작게 잡았다. 다음에는 배포 구성을 초반에 먼저 만들어 두고 기능을 올리는 순서로 하려고 한다.